AI Assignment 2- Learn!!

2/26/26

Assignment description

Instructions

1. Find a simple dataset on KaggleLinks to an external site.. From the dataset, choose a target attribute to try to classify.
2. Make a hypothesis for what you want to do with the data (e.g., can I classify whether an image has a stop sign in it).
3. Choose an ensemble classification learning method. You can choose any base classifier you'd like.
4. Choose at least two hyperparameters to try to tune for the method, and determine what range of values you will test
5. Learn! Use either OptTunaLinks to an external site. + SciKit LearnLinks to an external site. or the HyperparametersLinks to an external site. class in MATLAB to train various models.
   - You must use cross validation to test for the generalization error of your model
6. Write a short one page report describing
   - Your dataset
   - What ensemble method you chose
   - What base method you chose
   - What hyper parameters you chose, their ranges
   - The performance of your models using the different hyper parameters

Deliverables

Deliver a zip file of your code and a PDF of your report.

1. Your code
2. Your report

Grading

- 30% of your grade will be your report
- 40% of your grade will be your code
- 30% of your grade will be the peer reviews from the next pull request workshop.

AI Usage

AI Usage: Green light. You may use any AI coding tools. Just make sure you cite your tools, and be aware that you may loose points during your pull request session.

My Report

Github link: https://github.com/AshtonBeresford/AI_assignment2.git

I initially tried doing this assignment on a dataset of student grades. This would lead to failure due to my inexperience working with variables such as hyperparameters, and my attempt to use a random forest regressor. After being unable to develop a successful model for this dataset, I have gone back and selected a dataset that it more akin to this type of model, since I want to use one like this. which can be found at this link: https://www.kaggle.com/datasets/shree1992/housedata, which is a dataset on house prices.

Therefore this new model will also be a random forest regressor that will be used to experiment with this new dataset and try to predict the prices of houses based on their different attributes. I believe that I will be able to get better results with my newfound knowledge on this subject and also doing some research before going in on this second attempt. I will start out looking at independent values of: # bedrooms, # bathrooms, the # of square feet of living space, # floors, and a int from 1-5 that tells the condition of the property. I believe that the # of square feet will be the biggest determinant among the data fields.

The hyperparameters I'll be using to start out are min_samples_leaf (minimum samples required for leaf nodes) and n_estimators (number of trees). I'll be using Optuna to train these hyperparameters.

Initial results

As you can see in the images below, the model isn't perfectly set yet but its doing better than my original dataset. The R2 score got up to .2 which is quite weak but there's still room for improvement.

```
Best Parameters: {'n_estimators': 87, 'min_samples_leaf': 5}
Best R² Score:   0.19975655613062954
```

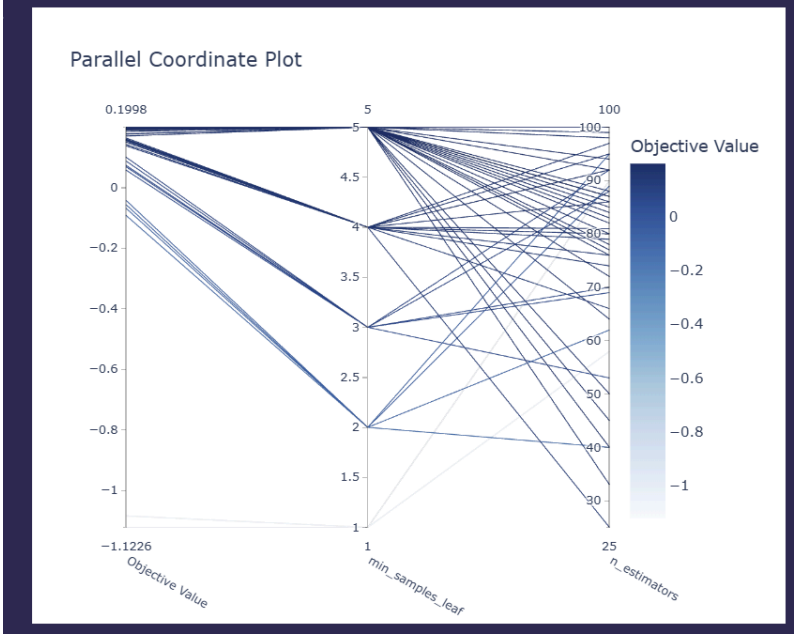

Figure 1: Initial attempt Best R^2 score and hyperparameter correlation plot

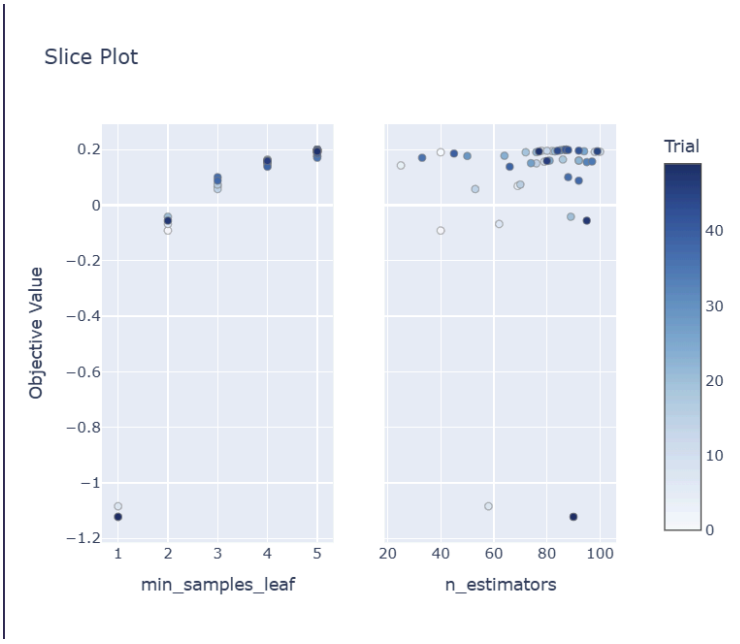

Figure 2: Initial attempt slice plot

Second iteration -- Results

I've messed with a lot of the values for this including different batches of hyperparameters and different independent variables impacting the function. While I have found improvement in the model, I have only managed to get the R2 score up to .31 range, which while still improved is still lackluster. I will go in and see Dr. Kropp to try and see any other potential changes I could make to the model. (After talking to Dr. Kropp he said it was probably fine to be done here, I'll probably look into it further on my own on a future date.)
Also, it appears that min_samples_leaf is consistently an impactful hyper-parameter, and when switching to using several hyperparameters the max_features becomes impactful. n_estimators and max_depth provided a little insight but were mostly not useful in this model. I tested other attributes but none of them had consistanly more impact that max_depth.

```
Best Parameters: {'n_estimators': 76, 'min_samples_leaf': 5, 'max_depth': 8, 'max_features': 'log2'}
Best R² Score:   0.2871680838233671
```

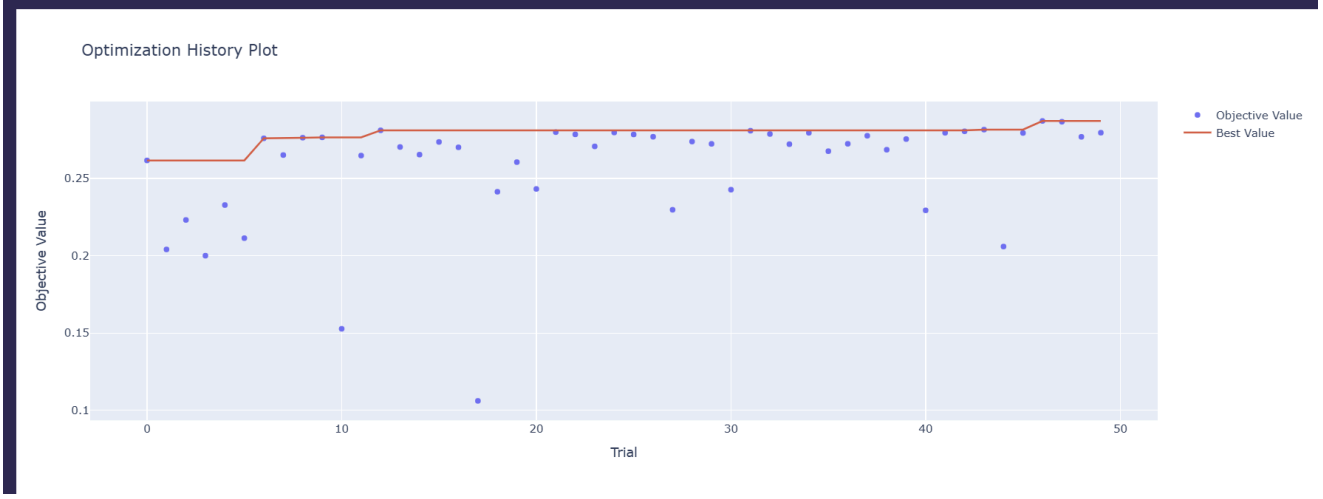

Figure 3: Second attempt best R^2 score and optimization plot history

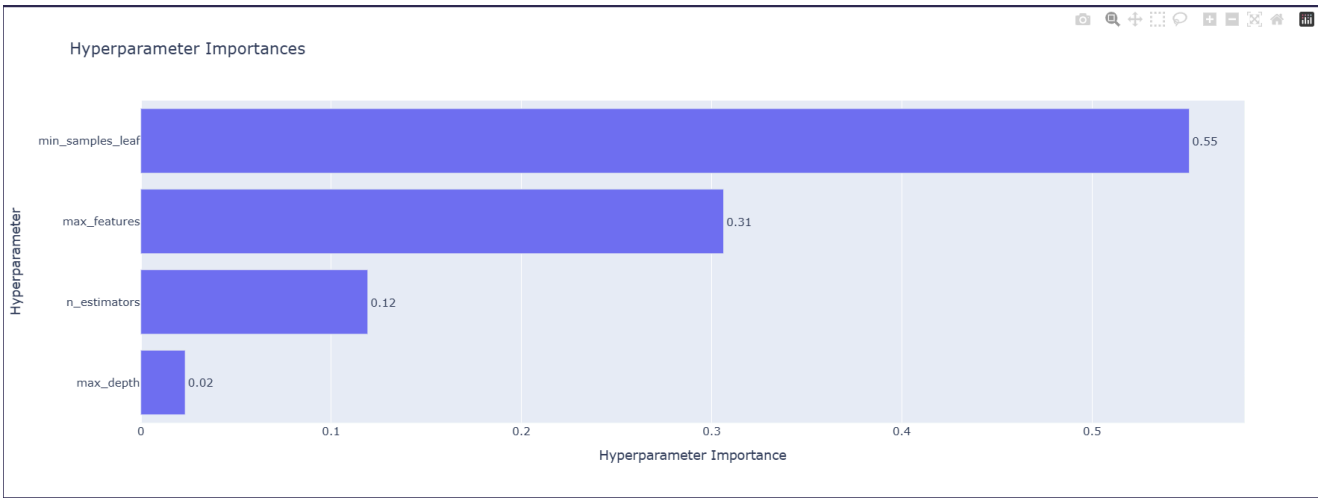

Figure 4: Second Attempt Hyperparameter Importance's

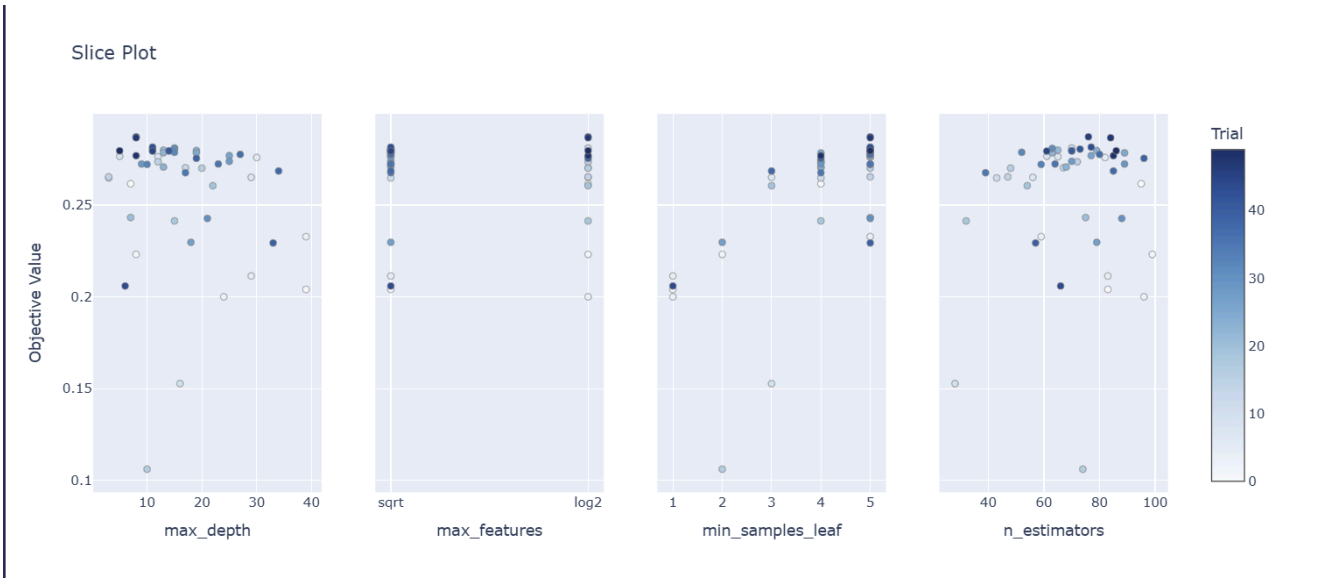

Figure 5: Second attempt Hyperparameter Slice Plot

Reflecting on Hypothesis

I was correct in stating that this second model of a random forest regressor would be better than my first model. As that one I couldn't get above a zero R^2 score, and for this model I've been able to get to a .31. However this model is still lower than I had hoped and predicted it would be. Therefore making it a bit lackluster as a final product.

One of the two hyperparameters I started out with proved to be very influential, the min_samples_leaf hyperparameter constantly improved the accuracy of the model, while the n_estimators hyperparameter wasn't influential. The ranges of the hyper parameters I used was: n_estimators 25-100, min_samples_leaf: 1-5, max_depth 3-40, and max_Features (sqrt, log2, None). I could've increased the min_samples_leaf number further to improve accuracy, but it would only result in small improvements that cap out. This can also all be seen down below in figure 6. Overall the models were better the more I used min_samples_leaf and max_features. The results containing the information about these different hyperparameters can be seen in figures 3-5 above.

Also you can see that I use cross-validation in the model below (cv=5) in figure 6. This image also shows the setup of my hyperparameters and the ranges I used for them.

```
def objective(trial):
    params = {
        'n_estimators'      : trial.suggest_int('n_estimators', 25, 100), # number of trees
        'min_samples_leaf'  : trial.suggest_int('min_samples_leaf', 1, 5), # minimum samples requi
        'max_depth'         : trial.suggest_int('max_depth', 3, 40),
        'max_features'     : trial.suggest_categorical('max_features', ['sqrt', 'log2', None])
    }
    model = RandomForestRegressor(**params, random_state=0) # sets up model with hyperparameters
    # trains the model 5 times with 4 folds for training and 1 fold for testing each time, cv=5 me
    score = cross_val_score(model, x_train, y_train, cv=5, scoring='r2', n_jobs=-1)
    return score.mean()
```

Figure 6: Setting up Model with Hyperparameters

The parameters I initially selected were influential: bedrooms, bathrooms, sqft_living, floors, condition as these gave me the best results, I only ended up adding one more parameter, yr_built, in order to improve the accuracy of the model.